Deep Research Agent

Backend API and LangGraph Flow Documentation

This document explains how the Python backend works, how the LangGraph workflow moves from a user question to a final published report, and how each Python file participates in the system.

Area	Summary
Backend API	FastAPI service in api.py exposes session, run start/resume, run snapshot, health, and translation endpoints.
LangGraph Orchestration	graph.py compiles a StateGraph with guardrails, history review, planning, search, reasoning, synthesis, human review, publishing, and persistence nodes.
Workflow Logic	nodes.py contains the actual node implementations, helper scoring/reranking utilities, report normalization, interrupt handling, and history persistence actions.
Contracts	state.py defines graph state keys. schemas.py defines strict Pydantic request, response, interrupt, and LLM output models.
External Tools	tools.py creates Tavily and Wikipedia LangChain tools. graph.py allows only tools approved by guardrails.

1. High-Level Architecture

The app has three major layers. The frontend calls FastAPI. FastAPI delegates execution to helper functions in app.py. Those helpers invoke a compiled LangGraph application from graph.py. The graph stores run state by thread_id and pauses at human review checkpoints by using LangGraph interrupts.

Layer	Main files	Responsibility
Presentation	react-frontend/, frontend.py	React is the main UI. frontend.py is an optional Streamlit UI for local exploration.
API boundary	api.py, schemas.py	Receives HTTP requests, validates payloads, converts graph state to frontend-friendly snapshots, and handles translation.
Run helpers	app.py	Creates graph instances, builds thread config, starts/resumes runs, and reads pending interrupts or current state.
Graph orchestration	graph.py, state.py	Defines LangGraph nodes, edges, routing decisions, checkpointer, shared store, and state shape.
Business logic	nodes.py, prompts.py, tools.py, history_store.py	Implements guardrails, history review, search, reasoning, synthesis, review, publishing, persistence, prompts, tools, and JSON history storage.

Typical request path

- The frontend requests POST /api/sessions to create a thread_id.
- The frontend posts a question to POST /api/runs/start.
- api.py calls start_research_run from app.py, passing a LangGraph config containing the thread_id.
- The graph executes until it completes or reaches an interrupt such as history review or draft approval.
- api.py returns a RunSnapshotResponse containing status, guardrails, metrics, interrupt payload, draft, search results, or final report.
- When the user chooses a decision, the frontend calls POST /api/runs/resume. app.py resumes the graph with Command(resume=...).

2. FastAPI Backend

api.py owns the HTTP API used by the React frontend. It creates one compiled research_app at module load time and reuses it across requests.

Endpoint	Purpose	Important behavior
GET /api/health	Health check.	Returns {status: ok}.
POST /api/sessions	Create a UI session.	Returns a unique thread_id such as ui-. This thread_id maps frontend interactions to one LangGraph run.
GET /api/runs/{thread_id}	Read current run snapshot.	Uses app.get_state and pending interrupts to classify status as idle, waiting_input, or completed.
POST /api/runs/start	Start a research run.	Validates StartResearchRequest, invokes the graph with question, user_id, max_iterations, and empty messages.
POST /api/runs/resume	Resume a paused run.	Sends the user decision and optional human feedback into LangGraph using Command(resume=...).
POST /api/translate	Translate final report text.	Uses deep_translator.GoogleTranslator and returns translated_text plus target_language.

The private helper _snapshot_for_thread is important because it adapts raw graph state into RunSnapshotResponse. It detects pending interrupts by reading task.interrupts, validates them as HistoryInterruptModel or ReviewInterruptModel,

and includes current run artifacts for the UI.

3. LangGraph Flow

graph.py builds and compiles the StateGraph. The graph uses ChatOpenAI(model='gpt-4o-mini'), OpenAIEmbeddings(model='text-embedding-3-small'), Tavily, Wikipedia, MemorySaver, and InMemoryStore.

Main path

```
START -> initialize_run_node -> evaluate_guardrails_node -> load_history_node -> history_review_node -> history_review_gate_node -> planner_node -> prepare_search_node -> tool_access_gate_node -> search_node -> tools -> capture_tool_results_node -> reason_node -> synthesise_node -> review_gate_node -> publish_node -> save_history_node -> END
```

Conditional branches

Decision point	Possible routes
After guardrails	continue -> load_history_node; blocked -> guardrail_block_node -> END.
After history gate	proceed_with_context -> planner_node; start_fresh_plan -> planner_node; reuse_existing -> reuse_existing_report_node -> save_history_node -> END.
After tool access gate	continue -> search_node; blocked -> guardrail_block_node -> END.
After search_node	If the LLM requested tools, route to ToolNode; otherwise go directly to reason_node.
After reason_node	CONTINUE -> prepare_search_node for another loop; DONE -> synthesise_node.
After review_gate_node	approved -> publish_node; edited -> apply_edit_node -> publish_node; rejected -> planner_node.

Interrupts and resumability

- history_review_gate_node pauses when similar or related history exists and asks the user whether to reuse, proceed with context, or start fresh.
- review_gate_node pauses before publishing and asks the user to approve, edit with feedback, or reject the draft.
- MemorySaver stores checkpoints per thread_id, so a later resume request can continue from the paused point.
- InMemoryStore stores shared research history while the backend process is alive; history_store.py also persists it to data/research_history.json.

4. What nodes.py Does

nodes.py is the workflow engine. It contains small graph nodes plus helper functions for cleaning inputs, scoring history, normalizing tool output, deduplicating/reranking evidence, computing metrics, validating drafts, and saving history.

Node	Role
initialize_run_node	Initializes default state keys such as iteration, decisions, reports, feedback, and metrics.
evaluate_guardrails_node	Sanitizes and assesses the question. Combines deterministic checks with structured LLM guardrail output.
guardrail_block_node	Creates a final blocked report when a request violates guardrail policy or no allowed tools are available.
load_history_node	Loads shared in-memory history and persisted JSON history, merges records, sorts newest first, and writes them into state.
history_review_node	Scores relevant prior records, asks the LLM whether the current question is similar, related, or new, and prepares history context.
history_review_gate_node	Raises a LangGraph interrupt when relevant history should be reviewed by the user.
reuse_existing_report_node	Uses the best matched prior report as final_report without doing new search.

Node	Role
planner_node	Creates 2 to 3 targeted search directions, optionally using relevant history as context.
prepare_search_node	Increments the loop iteration counter before a search round.
capture_tool_results_node	Reads ToolMessage outputs, normalizes Tavily/Wikipedia results, deduplicates evidence, and updates metrics.
reason_node	Decides whether the graph has enough evidence or should continue another search loop.
synthesise_node	Builds a structured DraftReportModel from accumulated evidence and history context.
review_gate_node	Raises the draft-review interrupt before publish.
apply_edit_node	Applies reviewer feedback to the draft summary before publishing.
publish_node	Turns the draft into a polished FinalReportModel with published_report text.
save_history_node	Persists completed final reports into shared store and data/research_history.json unless the run reused history.

5. State and Schemas

state.py defines the internal LangGraph state. schemas.py defines strict Pydantic models for HTTP payloads, graph snapshots, interrupts, and structured LLM outputs.

Concept	Defined in	How it is used
ResearchState	state.py	Shared dictionary carried through every LangGraph node. It includes question, user_id, messages, iteration counters, history, evidence, draft_report, final_report, and metrics.
messages	state.py	Annotated with add_messages so LangGraph appends messages across node updates.
Guardrail models	schemas.py	Constrain guardrail status, action, risk flags, allowed tools, explanation, and clarification prompt.
DraftReportModel and FinalReportModel	schemas.py	Validate generated draft/final reports and prevent malformed report structures from reaching the UI.
HistoryReviewModel	schemas.py	Constrains the LLM's memory comparison output to match_type, rationale, and relevant history references.
Interrupt models	schemas.py	HistoryInterruptModel and ReviewInterruptModel define the exact payload the frontend receives when the graph pauses.
RunSnapshotResponse	schemas.py	The main response shape returned by start, resume, and get snapshot endpoints.

Important state fields

- guardrails controls whether the run may proceed and which tools are allowed.
- past_topics holds loaded prior reports for history review.
- history_review and history_decision control whether prior work is reused, used as context, or ignored.
- retrieval_context stores selected history chunks; search_results stores normalized external evidence.
- research_plan guides tool queries; iteration and max_iterations control the search loop.
- draft_report, review_decision, human_feedback, and final_report represent the report lifecycle.

6. File-by-File Guide

File	Purpose
api.py	FastAPI app, CORS, HTTP endpoints, graph snapshot conversion, and translation endpoint.
app.py	Small adapter around LangGraph: create app, build config, build initial state, start/resume runs, inspect state and interrupts.
graph.py	Compiles the LangGraph StateGraph, creates LLM/embeddings/tools, wires every node and conditional route.
nodes.py	All workflow node implementations and helper logic for guardrails, history, evidence, reranking, reasoning, synthesis, review, publish, and persistence.
state.py	TypedDict definitions for internal graph state and nested report/evidence/history structures.
schemas.py	Pydantic validation for API requests/responses, LLM structured outputs, reports, interrupts, and snapshots.
prompts.py	Central prompt strings for planner, guardrails, history review, reasoner, synthesis, and publishing.
tools.py	Creates TavilySearch and WikipediaQueryRun tools consumed by graph.py's ToolNode.

File	Purpose
history_store.py	Loads, merges, sorts, and atomically saves JSON history in data/research_history.json.
validate_scenarios.py	HTTP-based validation runner that creates sessions, starts/resumes runs, and checks expected behavior.
sample_queries.py	Manual test query set for similar, related, and new history behavior.
demo.py	Minimal command-line graph demo with one canned research run and interrupt handling.
frontend.py	Optional Streamlit UI. The main frontend is react-frontend/.

7. History and Fresh Search Behavior

Published reports are saved as records containing question, report, user_id, and created_at. The backend keeps a shared in-memory store and also persists records to data/research_history.json. When a run starts, load_history_node merges both sources and sorts records newest first.

- merge_history_records deduplicates by question plus created_at.
- sort_history_records makes the newest saved record win when the same question appears multiple times.
- history_review_node filters history by deterministic relevance before the LLM sees prior records.
- If no relevant record passes the threshold, the graph treats the question as new.
- The user can choose reuse_existing, proceed_with_context, or start_fresh_plan when a relevant history interrupt appears.

8. Backend Data Flow Example

Step	Input	Output
1. Start	thread_id, question, user_id, max_iterations	Initial ResearchState in LangGraph.
2. Guardrails	question	Sanitized question, risk flags, allowed tools, proceed/revise/block decision.
3. History	question plus past_topics	new/similar/related assessment and optional interrupt.
4. Planning	question, guardrails, relevant history	2 to 3 search queries.
5. Search loop	plan, previous results, allowed tools	Tool calls, normalized evidence, metrics, reasoner DONE/CONTINUE.
6. Synthesis	evidence and history context	Structured draft report.
7. Human review	draft report	approved, edited, or rejected decision.
8. Publish	approved draft	FinalReportModel and polished published_report text.
9. Save	final report	History record in memory and JSON for future comparisons.

9. Developer Notes

- Environment variables: OPENAI_API_KEY is required for ChatOpenAI and embeddings. TAVILY_API_KEY is required for Tavily search.
- Backend command: uvicorn api:app --reload.
- Frontend command from react-frontend/: npm run dev.
- Validation command: python validate_scenarios.py --base-url http://localhost:8000/api.
- When debugging stale history, inspect data/research_history.json and the history_review interrupt payload returned by /api/runs/{thread_id}.
- When debugging search quality, inspect search_results, retrieval_context, run_metrics, and the allowed_tools selected by guardrails.